

A large orange circle is positioned on the left side of the slide, partially cut off by the edge.

Git e GitHub para Desenvolvimento de Sistemas: Um Guia Abrangente

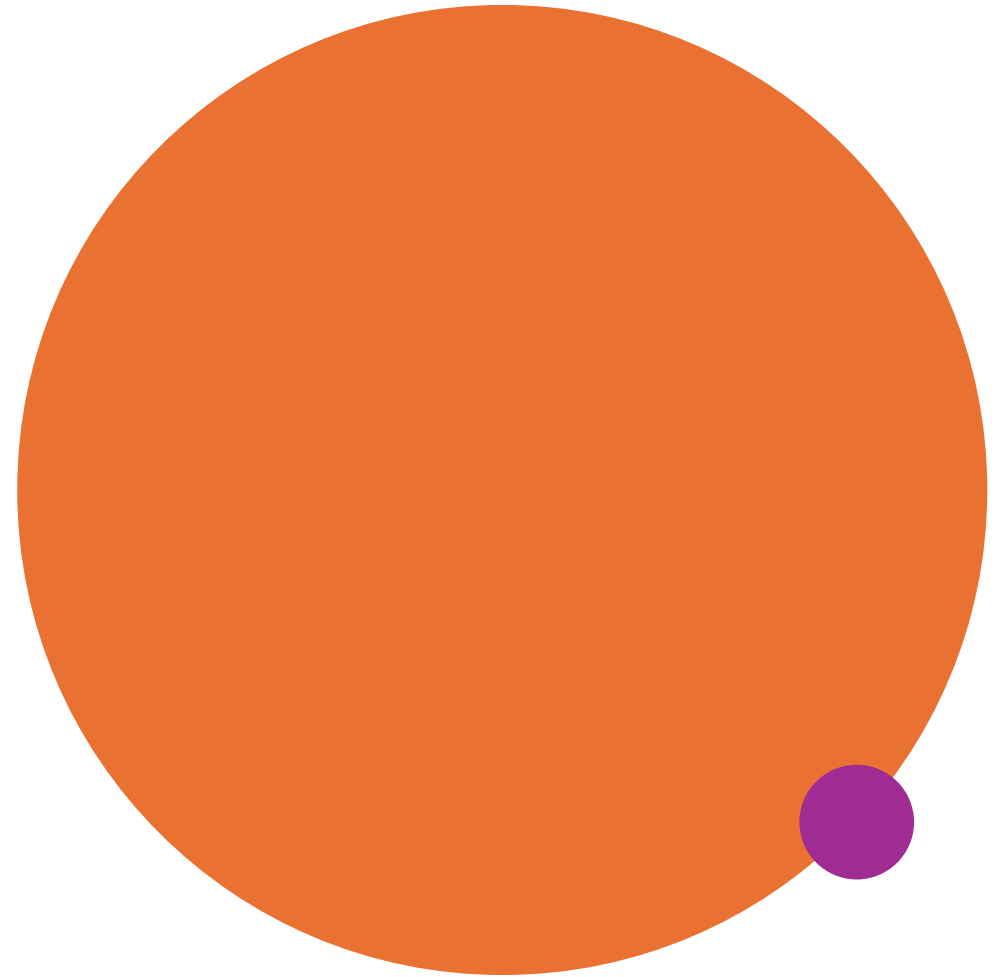
- Instrutor: **Djalma Batista Barbosa Junior**
- E-mail: djalma.batista@fiemg.com.br



Introdução ao Controle de Versão

- O controle de versão é uma prática essencial na engenharia de software, utilizada para rastrear e gerenciar alterações no código fonte ao longo do tempo. Sistemas de controle de versão permitem que equipes de desenvolvimento trabalhem de forma mais eficiente, especialmente em ambientes acelerados, como os de DevOps, reduzindo o tempo de desenvolvimento e aumentando a taxa de sucesso nas implementações. Esses sistemas mantêm um registro detalhado de todas as mudanças no código, o que facilita a correção de erros ao permitir que os desenvolvedores voltem a versões anteriores e comparem alterações, minimizando impactos na equipe. Além disso, o controle de versão protege o código de falhas catastróficas e da degradação gradual causada por erros humanos.

- 
- Existem três tipos principais de sistemas de controle de versão: local, centralizado e distribuído. Os sistemas locais armazenam alterações no computador do usuário, sendo simples, mas limitados para equipes modernas. Os centralizados possuem um repositório único, o que facilita a colaboração, mas cria um ponto único de falha. Já os sistemas distribuídos, como o Git, oferecem maior flexibilidade e robustez, pois cada usuário tem uma cópia completa do código e seu histórico, permitindo trabalho offline e estratégias avançadas de ramificação e mesclagem. Esses sistemas são especialmente úteis para equipes que trabalham simultaneamente em diferentes partes do código, prevenindo conflitos e garantindo a compatibilidade das alterações



Introdução ao Git

- Git é um sistema de controle de versão distribuído (DVCS) amplamente utilizado para rastrear alterações em arquivos, especialmente em código fonte, durante o desenvolvimento de software ¹. Criado por Linus Torvalds em 2005, o Git se tornou uma ferramenta essencial para desenvolvedores em todo o mundo, facilitando a colaboração eficiente, o rastreamento de mudanças e o gerenciamento de diferentes versões de um projeto.

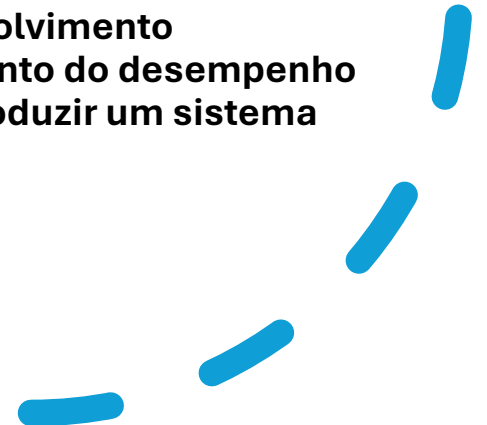


Histórico do Git

- O Git foi criado em 2005 por Linus Torvalds como parte do desenvolvimento do kernel do Linux. Antes disso, a equipe do kernel utilizava o BitKeeper para controle de versão, uma ferramenta que era usada gratuitamente desde 2002. No entanto, em 2005, a licença gratuita do BitKeeper foi revogada devido a uma disputa de licenciamento, o que levou a equipe do kernel, e especialmente Torvalds, a desenvolver seu próprio sistema de controle de versão.
- Torvalds buscava um sistema distribuído, rápido e capaz de gerenciar múltiplas ramificações em um único repositório, permitindo um desenvolvimento não linear. O Git foi projetado para atender a essas necessidades. Inicialmente, ele contava com comandos básicos, chamados de "plumbing", que precisavam ser combinados para realizar tarefas. Com o tempo, comandos de nível superior, conhecidos como "porcelain", foram adicionados para simplificar e agilizar o uso do Git, tornando-o mais acessível e eficiente.

Objetivos de Design do Git

- O Git foi projetado com foco em velocidade, integridade de dados e suporte para fluxos de trabalho distribuídos e não lineares, com a capacidade de gerenciar milhares de ramificações paralelas em diferentes computadores ¹⁴. Assim como outros sistemas de controle de versão distribuídos, o Git mantém uma cópia local de todo o repositório, incluindo histórico e capacidades de rastreamento de versão,
- independentemente do acesso à rede ou a um servidor central.
- Para seu critério de design, Torvalds especificou que a aplicação de patches não deveria levar mais de três segundos, e adicionou mais três objetivos: tomar o Concurrent Versions System (CVS) como um exemplo do que não fazer, suportar um fluxo de trabalho distribuído semelhante ao BitKeeper e incluir fortes salvaguardas contra corrupção, seja acidental ou maliciosa . O design do Git é uma síntese da experiência de Torvalds com o Linux na manutenção de um grande projeto de desenvolvimento distribuído, juntamente com seu profundo conhecimento do desempenho do sistema de arquivos e a necessidade urgente de produzir um sistema funcional em curto prazo .



Git vs. Outros Sistemas de Controle de Versão

- Embora existam outros sistemas de controle de versão, como Subversion (SVN), Mercurial e Perforce, o Git se destaca por sua arquitetura distribuída, poderosos recursos de ramificação e mesclagem, alto desempenho, flexibilidade e ampla adoção .
- Em contraste com o SVN, que é centralizado e depende de um único servidor, o Git é distribuído, o que significa que cada desenvolvedor possui uma cópia completa do repositório, incluindo todo o histórico. Isso proporciona maior flexibilidade, confiabilidade e controle sobre o gerenciamento do código. As capacidades de ramificação e mesclagem do Git são consideradas superiores às do SVN, oferecendo mais opções e resolução de conflitos mais fácil. Além disso, o Git geralmente apresenta melhor desempenho que o SVN, especialmente em projetos grandes, devido ao seu armazenamento eficiente e operações locais.

- 
- Embora o Mercurial seja considerado mais fácil para iniciantes, o Git possui uma comunidade maior e um ecossistema mais rico, tornando-o a escolha preferida para muitas equipes. O Perforce é conhecido por lidar com bases de código muito grandes, mas o modelo de ramificação e mesclagem do Git é mais flexível e amigável.
 - A arquitetura distribuída do Git permite que os desenvolvedores trabalhem offline e façam commits incrementais. Cada desenvolvedor tem seu próprio repositório, eliminando um ponto único de falha.
- 

Fluxo de Trabalho Básico do Git

- O fluxo de trabalho básico do Git envolve a inicialização de um repositório, a adição de arquivos, a realização de commits para salvar as alterações, a verificação do status do repositório e a visualização do histórico de commits .



Inicializando um Repositório Git (git init)

- O comando `git init` é usado para criar um novo repositório Git. Ele pode transformar um projeto existente e não versionado em um repositório Git ou iniciar um novo repositório vazio. A maioria dos outros comandos do Git só funciona dentro de um repositório inicializado, o que faz do `git init` geralmente o primeiro comando a ser executado em um novo projeto. Quando o `git init` é executado, uma subpasta oculta chamada `.git` é criada no diretório de trabalho atual, armazenando todos os metadados necessários para o repositório.
- Para inicializar um repositório Git em um diretório específico, pode-se usar o comando `git init <diretório>`. Já para criar um repositório "bare" (sem diretório de trabalho), que é comumente utilizado para repositórios remotos centrais, o comando é `git init --bare`.

Clonando um Repositório Existente (git clone)

- Para obter uma cópia de um repositório Git existente, como um projeto no qual se deseja contribuir, o comando utilizado é `git clone`. Diferentemente de outros VCS, o `git clone` recebe uma cópia completa de quase todos os dados que o servidor possui, incluindo cada versão de cada arquivo para todo o histórico do projeto. O comando básico para clonar um repositório é `git clone <url>`. Para clonar o repositório em um diretório com um nome diferente do padrão, especifica-se o novo nome como um argumento adicional: `git clone <url> <novo_diretório>`.

Adicionando Arquivos ao Staging Area (git add)

- O comando `git add` adiciona uma mudança no diretório de trabalho ao staging area. Ele informa ao Git que se deseja incluir atualizações em um arquivo específico no próximo commit. O `git add` não afeta o repositório de forma significativa; as mudanças não são realmente registradas até que se execute o comando `git commit`.
- Para adicionar um único arquivo, utiliza-se `git add <nome_do_arquivo>`. Para adicionar todos os arquivos modificados e novos no diretório atual e subdiretórios, usa-se `git add .` A opção `-A` ou `--all` adiciona todas as mudanças em todo o repositório. O comando `git add -p` inicia uma sessão de staging interativa, permitindo escolher partes específicas de um arquivo para adicionar ao próximo commit.

Realizando Commits para Salvar Alterações (git commit)

O comando `git commit` captura um snapshot das mudanças atualmente staged do projeto. Commits podem ser pensados como versões "seguras" de um projeto; o Git nunca os alterará a menos que explicitamente solicitado. Antes da execução de `git commit`, o comando `git add` é usado para promover ou "stage" as mudanças que serão armazenadas no commit.

O comando básico para realizar um commit é `git commit`. Isso abrirá um editor de texto para inserir uma mensagem de commit. Para especificar a mensagem de commit diretamente na linha de comando, usa-se a opção `-m`: `git commit -m "sua mensagem de commit aqui"`. A opção `-a` com o commit (`git commit -a` ou `git commit -am "mensagem"`) commita automaticamente todas as mudanças nos arquivos rastreados, pulando a etapa do `git add`.

Verificando o Status do Repositório (git status)

- O comando `git status` exibe o estado do diretório de trabalho e do staging area. Ele permite visualizar quais mudanças foram staged, quais não foram e quais arquivos não estão sendo rastreados pelo Git. A saída do `git status` também inclui instruções relevantes para staging/unstaging de arquivos.

Visualizando o Histórico de Commits (git log)

- O comando `git log` exibe os snapshots que foram commitados, permitindo visualizar o histórico do projeto, filtrá-lo e buscar por alterações específicas. Diferente do `git status`, que analisa o diretório de trabalho e a área de staging, o `git log` opera exclusivamente no histórico de commits.
- Entre as opções mais comuns estão: `git log --oneline`, que exibe cada commit em uma única linha; `git log --stat`, que mostra quais arquivos foram modificados e o número de linhas adicionadas ou removidas; `git log -p`, que exibe o patch completo (diff) de cada commit; `git log --author="<padrão>"`, que filtra os commits por um autor específico; e `git log --grep="<padrão>"`, que busca commits com mensagens que correspondem ao padrão especificado.

Comando	Descrição	Exemplo de Uso
git init	Inicializa um novo repositório Git local.	git init
git clone	<u>Clona</u> um repositório Git existente de uma URL.	git clone https://...
git add	Adiciona arquivos ao staging area.	git add arquivo.txt, git add .
git commit	Salva as alterações no repositório local.	git commit -m "Mensagem"
git status	Exibe o estado do <u>diretório</u> de trabalho e do staging area.	git status
git log	Mostra o histórico de commits do repositório.	git log, git log --oneline
git diff	Exibe as diferenças entre o <u>diretório</u> de trabalho, o staging area e o histórico.	git diff, git diff --staged

Branching e Merging no Git

- O branching e o merging são funcionalidades robustas do Git, que possibilitam o desenvolvimento paralelo e a integração de diversas linhas de trabalho.



Criando Branches (git branch)

- Um branch no Git representa uma linha de desenvolvimento independente, funcionando como uma abstração para o processo de edição, staging e commit. Eles podem ser vistos como uma forma de criar um novo diretório de trabalho, uma área de staging e um histórico de projeto separados. Cada novo commit é registrado no histórico do branch atual, criando uma bifurcação no histórico do projeto.
- Para listar os branches existentes, utiliza-se o comando `git branch`, sendo que o branch atual é geralmente indicado por um asterisco. Para criar um novo branch, o comando utilizado é `git branch <nome_do_branch>`.

Trocando de Branch (git checkout ou git switch)

Para mudar para um branch específico e começar a trabalhar nele, o comando utilizado é git checkout <nome_do_branch>. Uma forma mais rápida de criar um novo branch e já alternar para ele é usando git checkout -b <novo_nome_do_branch>.

Em versões mais recentes do Git (2.23+), o comando git switch foi introduzido como uma opção mais intuitiva para trocar de branches: git switch <nome_do_branch>. Para criar e mudar para um novo branch com git switch, o comando é git switch -c <novo_nome_do_branch>.

Mesclando Branches (git merge)

- A mesclagem é o método do Git para reunir um histórico de desenvolvimento que foi dividido. O comando `git merge` permite integrar as linhas de desenvolvimento independentes, criadas com `git branch`, em um único branch.
- Para mesclar um branch (`<branch_a_ser_mesclado>`) no branch ativo, primeiro é necessário alternar para o branch de destino usando `git checkout <branch_de_destino>` e, em seguida, executar o comando `git merge <branch_a_ser_mesclado>`.
- O Git realiza a mesclagem de duas formas principais: fast-forward e merge de três vias. O fast-forward ocorre quando há um caminho linear do branch atual até o branch de destino, e o Git simplesmente move o ponteiro do branch atual para a ponta do branch de destino. Já o merge de três vias é utilizado quando os branches divergiram, criando um novo commit de merge que combina as alterações de ambos os branches, usando um ancestral comum como base.

Trabalhando com Repositórios Remotos

- Repositórios remotos são cópias do seu projeto armazenadas em servidores externos, como GitHub, GitLab ou Bitbucket. Eles são essenciais para facilitar a colaboração entre desenvolvedores, permitindo o compartilhamento de código e o acompanhamento das alterações feitas por cada membro da equipe



git branch	Lista os branches locais; <u>cria</u> , <u>renomeia</u> ou <u>exclui</u> branches.	git branch, git branch novo-branch	
git checkout	Troca para um branch existente; cria e troca para um novo branch.	git checkout main, git checkout -b novo-branch	
git switch	Troca para um branch existente; cria e troca para um novo branch (mais recente).	git switch main, git switch -c novo-branch	
git merge	Mescla as alterações de um branch em outro.	git checkout main, git merge feature-branch	
git branch -d	Exclui um branch local (apenas se já tiver sido mesclado)	git branch -d feature-branch	

Adicionando um Repositório Remoto (git remote add)

- Para vincular seu repositório local a um repositório remoto, o comando utilizado é `git remote add <nome> <url>`. O `<nome>` serve como um apelido para o remoto (normalmente chamado de origin), e a `<url>` é o endereço do repositório remoto. Para visualizar os remotos configurados, pode-se usar o comando `git remote -v`.

Enviando Alterações para o Remoto (git push)

- O comando `git push` é utilizado para enviar as alterações do seu repositório local para um repositório remoto. A sintaxe básica é `git push <remoto> <branch>`. Por exemplo, para enviar o branch `main` para o remoto `origin`, usa-se `git push origin main`. A opção `-u` na primeira vez que um novo branch é enviado (com `git push -u origin <novo_branch>`) configura o rastreamento do branch remoto, permitindo que, nas próximas vezes, apenas `git push` seja necessário para enviar as alterações.

Obtendo Alterações do Remoto (git fetch e git pull)

- O comando `git fetch` baixa commits e objetos do repositório remoto, mas não aplica essas alterações ao seu branch local. Por outro lado, o comando `git pull` combina `git fetch` com `git merge` ou `git rebase`, integrando as mudanças remotas diretamente no seu branch local. A sintaxe básica é `git pull <remoto> <branch>`. Por exemplo, ao usar `git pull origin main`, as alterações do branch `main` do remoto `origin` são baixadas e mescladas no branch local atual.

Comando	Descrição	Exemplo de Uso
git remote add	Adiciona um novo repositório remoto.	git remote add origin https://...
git remote -v	Lista os repositórios remotos configurados.	git remote -v
git push	<u>Envia</u> commits locais para um repositório remoto.	git push origin main
git fetch	Baixa commits e objetos de um repositório remoto sem mesclar.	git fetch origin
git pull	Baixa commits e objetos de um repositório remoto e os mescla no branch local.	git pull origin main

Relação entre Git e GitHub

- É essencial destacar que Git e GitHub não são a mesma coisa. O Git é uma ferramenta de controle de versão de código aberto, desenvolvida em 2005, enquanto o GitHub é uma plataforma fundada em 2008 que oferece serviços de hospedagem para repositórios Git. É possível utilizar o Git sem o GitHub, mas o GitHub depende do Git para funcionar. O GitHub aproveita a tecnologia do Git para rastrear e gerenciar as mudanças nos arquivos armazenados em seus repositórios.

Principais Funcionalidades do GitHub

O GitHub oferece uma variedade de funcionalidades que simplificam a colaboração e o gerenciamento de projetos de software.

Repositórios: São espaços onde o código do projeto e seu histórico de alterações são armazenados. O GitHub permite a criação de repositórios públicos (acessíveis a todos) ou privados (com acesso limitado).

Issues: São ferramentas para rastrear tarefas, melhorias, relatos de bugs e feedback relacionados ao projeto. Elas permitem discussões detalhadas sobre aspectos específicos e podem ser atribuídas a membros da equipe.

Pull Requests: Permitem propor mudanças específicas em um branch e iniciar discussões com colaboradores antes de mesclar as alterações ao branch principal. Os pull requests são fundamentais para processos de revisão de código.

Colaboração: O GitHub facilita a colaboração entre desenvolvedores, permitindo que trabalhem simultaneamente no mesmo projeto sem conflitos. Por meio de pull requests, revisões de código e discussões em issues, as equipes podem garantir a qualidade do código e compartilhar conhecimento de forma eficiente.

Fluxo de Trabalho do GitHub

- O fluxo de trabalho comum no GitHub inclui a criação de um repositório, a realização de alterações em branches separados, a abertura de pull requests para revisão e a mesclagem das mudanças aprovadas no branch principal.



Criando um Repositório no GitHub

- Um novo projeto geralmente começa com a criação de um repositório no GitHub. Isso pode ser feito diretamente pela interface web da plataforma, onde é possível definir o nome do repositório, adicionar uma descrição e escolher se ele será público ou privado. Depois que o repositório remoto é criado, ele pode ser clonado para o ambiente local utilizando o comando `git clone <url_do_repositório>`.

Fazendo Alterações em Branches

- Para desenvolver uma nova funcionalidade ou corrigir um bug, é recomendado criar um novo branch a partir do branch principal (normalmente main ou master). Isso mantém as alterações isoladas, evitando que interfiram no código principal até que estejam finalizadas e revisadas. As mudanças são realizadas, adicionadas à área de staging com `git add` e commitadas com `git commit` no branch da funcionalidade.

Abrindo um Pull Request

- Depois de fazer os commits com as mudanças no branch da funcionalidade, o próximo passo é enviar esse branch para o repositório remoto usando `git push origin <nome_do_branch>` e, em seguida, abrir um pull request no GitHub. O pull request avisa os colaboradores sobre as alterações sugeridas e permite que revisem o código, deixem comentários e proponham ajustes

Mesclando o Pull Request

- Após as alterações no pull request serem revisadas e aprovadas pelos colaboradores, elas podem ser mescladas no branch principal. Normalmente, isso é feito pela interface web do GitHub, onde um botão "Merge pull request" está disponível. Depois da mesclagem, o branch da funcionalidade pode ser removido, e as mudanças serão incorporadas ao branch principal



Boas Práticas de Versionamento

- dotar boas práticas de versionamento com Git e GitHub é essencial para assegurar um fluxo de trabalho eficiente, colaborativo e para preservar a integridade do histórico do projeto.



Convenções de Mensagens de Commit

- Criar mensagens de commit claras e descritivas é crucial para entender o histórico do projeto e as razões por trás de cada alteração. Uma boa mensagem de commit geralmente apresenta um resumo breve e direto na primeira linha, seguido por uma linha em branco e, se necessário, um corpo com detalhes adicionais sobre o contexto e a motivação da mudança. É recomendado usar o modo imperativo e o tempo presente no resumo (por exemplo, "Corrigir bug Y" em vez de "Corrigido bug Y"). Adotar commits convencionais (como feat:, fix:, docs:) pode simplificar a geração automática de changelogs e tornar mais fácil identificar o tipo de alteração realizada.



Estratégias de Branching

- Adotar uma estratégia de branching consistente é essencial para organizar o trabalho e gerenciar diferentes versões do projeto. Algumas abordagens populares incluem:
- Feature Branch Workflow: O desenvolvimento de novas funcionalidades ocorre em branches dedicados, que são mesclados ao branch principal após revisão.
- Gitflow Workflow: Utiliza dois branches principais (main para releases e develop para integração de funcionalidades), além de branches específicos para funcionalidades, releases e correções de emergência. É ideal para projetos com ciclos de lançamento planejados.
- GitHub Flow: Uma estratégia mais simplificada, com um único branch principal (main) e branches de funcionalidade de curta duração, que são mesclados de volta ao main após revisão e testes. É especialmente adequado para ambientes de entrega contínua.

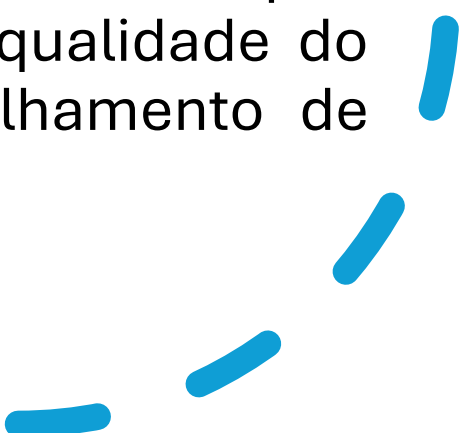
Lidando com Conflitos de Merge.

- Conflitos de merge são comuns no desenvolvimento colaborativo. Quando ocorrem, o Git sinaliza os arquivos afetados e exige intervenção manual para resolvê-los. A resolução envolve abrir o arquivo, localizar as seções em conflito (marcadas por indicadores como <<<<<<, ===== e >>>>>>), decidir quais alterações manter e ajustar o arquivo conforme necessário. Após a resolução, o arquivo deve ser adicionado à área de staging (git add) e o commit de merge finalizado (git commit).



Outras Boas Práticas



- 
- Fazer commits com frequência para manter as alterações pequenas e fáceis de gerenciar.
 - Atualizar o branch local regularmente com as mudanças do repositório remoto (git pull) para evitar divergências significativas e conflitos.
 - Usar o arquivo .gitignore para excluir arquivos desnecessários do controle de versão, como arquivos de build ou temporários.
 - Realizar revisões de código por meio de pull requests no GitHub, garantindo a qualidade do código e promovendo o compartilhamento de conhecimento.
- 

Conceitos e Ferramentas Avançadas do Git

- **O Git oferece diversos conceitos e ferramentas avançadas que podem otimizar o fluxo de trabalho e fornecer maior controle sobre o histórico do projeto.**



Rebasing (git rebase)

- O rebase é uma maneira de integrar mudanças de um branch em outro, reposicionando ou combinando uma série de commits em um novo branch base. Ao contrário do merge, o rebase cria um histórico mais linear, dando a impressão de que os commits do branch de origem foram feitos diretamente sobre o branch de destino. O rebase interativo (git rebase -i) permite editar, reordenar ou mesclar commits antes de aplicá-los ao novo branch base.

Resetting (git reset)

- O comando `git reset` é usado para desfazer alterações no diretório de trabalho, no staging area e no histórico de commits. Existem três tipos principais de reset: `--soft` (mantém as alterações no staging area), `--mixed` (remove as alterações do staging area, mas as mantém no diretório de trabalho) e `--hard` (remove as alterações do staging area e do diretório de trabalho, descartando-as).



Stashing (git stash)

- O stashing permite salvar temporariamente alterações não commitadas e limpar o diretório de trabalho para o último commit. Isso é útil quando se precisa mudar de branch rapidamente sem commitar trabalho incompleto. O comando `git stash pop` reaplica as alterações salvas ao diretório de trabalho.

Clientes GUI para Git

- Além da linha de comando, existem diversos clientes gráficos (GUIs) para Git que podem facilitar a visualização e o gerenciamento de repositórios. Alguns exemplos populares incluem GitHub Desktop, Sourcetree e GitKraken. Esses clientes oferecem interfaces intuitivas para realizar operações comuns do Git, como commitar, fazer push, pull, criar branches e resolver conflitos.

Integração com IDEs

- A maioria dos Ambientes de Desenvolvimento Integrados (IDEs) modernos possui integração embutida com o Git, permitindo que os desenvolvedores realizem operações do Git diretamente do IDE. Isso pode simplificar o fluxo de trabalho, pois não é necessário alternar entre o IDE e a linha de comando para realizar tarefas de controle de versão. Exemplos de IDEs com boa integração Git incluem IntelliJ IDEA, VS Code e Eclipse.

Aplicações Reais de Git e GitHub

- Git e GitHub são ferramentas indispensáveis no desenvolvimento de software moderno, com uma ampla gama de aplicações.



Colaboração em Projetos de Código Aberto

- O GitHub se tornou o padrão de fato para projetos de código aberto. Ele facilita a contribuição de desenvolvedores de todo o mundo através de funcionalidades como forking e pull requests, permitindo que a comunidade colabore na construção e melhoria de softwares.



Desenvolvimento de Software em Equipe

- **Git e GitHub são essenciais para gerenciar mudanças de código e facilitar a colaboração em equipes de desenvolvimento de todos os tamanhos. Eles permitem que múltiplos desenvolvedores trabalhem no mesmo projeto simultaneamente sem conflitos, rastreando cada alteração e permitindo a reversão para versões anteriores se necessário.**

Processos de Revisão de Código

- Os pull requests no GitHub permitem processos organizados de revisão de código, possibilitando que os membros da equipe forneçam feedback sobre as mudanças sugeridas antes que sejam incorporadas ao código principal. Isso ajuda a melhorar a qualidade do código e a identificar possíveis problemas de forma antecipada.

Integração Contínua e Entrega Contínua (CI/CD)

- Git e GitHub se integram de forma eficiente com pipelines de CI/CD, automatizando a construção, os testes e a implantação de software. As alterações commitadas no repositório Git podem acionar automaticamente processos de build e teste, assegurando que o software seja entregue de maneira ágil e confiável.

Controle de Versão para Projetos Não Codificados

- Embora sejam principalmente voltados para código, Git e GitHub também podem ser usados para rastrear mudanças em outros tipos de arquivos, como documentação, recursos de design e configurações. Isso assegura que todos os elementos do projeto sejam versionados e gerenciados de maneira consistente.



GitHub Copilot e Desenvolvimento Assistido por IA

- Ferramentas como o GitHub Copilot empregam inteligência artificial para ajudar desenvolvedores com sugestões de código, explicações e outras funcionalidades, utilizando o contexto dos repositórios Git. Isso pode impulsionar a produtividade e simplificar o processo de escrita de código.



Conclusão

- A adoção de Git e GitHub no desenvolvimento de sistemas oferece inúmeros benefícios, desde o controle preciso das alterações no código até a facilitação da colaboração em equipe e a integração com fluxos de trabalho modernos de CI/CD. O controle de versão é uma prática essencial para desenvolvedores individuais e equipes, permitindo trabalhar de forma mais eficiente, reduzir erros e manter um histórico completo e confiável do desenvolvimento do projeto. É altamente recomendável que os estudantes pratiquem o uso dos comandos Git e explorem as funcionalidades do GitHub para se familiarizarem com essas ferramentas poderosas e indispensáveis no cenário atual do desenvolvimento de software.